

Cảm nhận về đẹp của ngẫu nhiên

Gu Yan

Trường Trung học số 1 Trung Sơn, Quảng Đông

Trại đông Tin học toàn quốc, 2008

Nguồn gốc: Cảm nhận về đẹp của ngẫu nhiên: bàn về ứng dụng tư tưởng ngẫu nhiên hóa trong bài toán hình học

IOI China candidate team papers / 2008 / Day 1 / Gu Yan / randomized geometry.doc

Tóm tắt

Trong vài năm gần đây, các bài hình học trong thi tin học không còn chỉ là cuộc thi viết lại những cấu trúc kinh điển. Nhiều bài có thể được giải gọn hơn nếu đưa ngẫu nhiên vào thứ tự xử lý, hoặc dùng một quá trình tìm kiếm ngẫu nhiên có kiểm soát. Bài viết giới thiệu hai nhóm kỹ thuật chính: thuật toán gia tăng ngẫu nhiên và mô phỏng tôi luyện. Qua các ví dụ đường tròn bao nhỏ nhất, quy hoạch tuyến tính ba chiều, *Run Away*, *Empire Strikes Back* và *Laser Tank*, tác giả nhấn mạnh ưu điểm đơn giản, nhanh, linh hoạt và dễ song song hóa của tư tưởng ngẫu nhiên hóa.

Từ khóa: ngẫu nhiên hóa; thuật toán gia tăng ngẫu nhiên; mô phỏng tôi luyện; điều chỉnh.

1 Khái quát về thuật toán ngẫu nhiên

Thuật toán ngẫu nhiên là loại thuật toán chấp nhận dữ liệu đầu vào đồng thời đưa yếu tố ngẫu nhiên vào quá trình chọn bước tiếp theo. Vì vậy với cùng một đầu vào, hai lần chạy có thể đi qua các trạng thái khác nhau, có thời gian khác nhau, hoặc trong một số mô hình còn cho đáp án xấp xỉ hơi khác nhau.

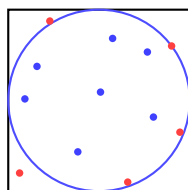
Ưu điểm thường gặp của loại thuật toán này là đơn giản, nhanh, linh hoạt và dễ song song hóa. Có thể xem ngẫu nhiên hóa như một cách cân bằng giữa thời gian, bộ nhớ và độ chính xác. Bản gốc chia các thuật toán ngẫu nhiên thường dùng thành bốn nhóm sau.

1.1 Thuật toán xác suất số trị

Thuật toán xác suất số trị thường dùng để giải các bài toán số. Kết quả thu được thường là nghiệm xấp xỉ, và khi thời gian tính tăng thì độ chính xác cũng tăng. Trong nhiều tình huống, nghiệm chính xác là không thể tính hoặc không cần thiết; một nghiệm xấp xỉ đủ tốt đã đáp ứng yêu cầu.

Ví dụ kinh điển là ước lượng π . Lấy ngẫu nhiên n điểm trong hình vuông ngoại tiếp đường tròn đơn vị. Nếu k điểm rơi vào trong đường tròn, ta có xấp xỉ

$$\pi \approx \frac{4k}{n}.$$



hình vuông: diện tích 4
đường tròn: diện tích π

Hình 1: Lấy mẫu Monte Carlo để ước lượng π : tỉ lệ điểm nằm trong đường tròn xấp xỉ tỉ lệ diện tích.

1.2 Las Vegas

Thuật toán Las Vegas không trả lời sai. Một khi thuật toán tìm được nghiệm, nghiệm đó chắc chắn đúng. Điểm ngẫu nhiên nằm ở thời gian chạy: đôi khi thuật toán chưa tìm được nghiệm trong thời gian ngắn, nhưng nếu cho chạy lâu hơn thì xác suất thành công tăng lên.

1.3 Monte Carlo

Monte Carlo cũng được đặt theo tên một thành phố cờ bạc. Theo cách dùng phổ biến, có hai dạng.

Dạng thứ nhất là kiểm định Monte Carlo. Thuật toán luôn đưa ra câu trả lời trong thời gian kiểm soát được, nhưng có thể sai với xác suất nhỏ. Lỗi thường được thiết kế một phía: chẳng hạn nếu đáp án thật là No thì thuật toán luôn trả lời No, còn nếu đáp án thật là Yes thì đôi khi nó trả lời No. Khi tăng số lần thử, xác suất trả lời đúng tăng lên. Kiểm định nguyên tố Miller–Rabin và Rabin–Karp cho so khớp chuỗi là các ví dụ quen thuộc.

Dạng thứ hai là lấy mẫu Monte Carlo. Ta xây dựng một mô hình xác suất cho bài toán, mô phỏng nhiều lần theo mô hình đó, rồi dùng kỳ vọng mẫu làm nghiệm xấp xỉ. Phụ lục A tóm tắt ba bước cơ bản của cách làm này. Mô phỏng tôi luyện ở phần sau cũng dùng tinh thần lấy mẫu Monte Carlo.

1.4 Sherwood

Thuật toán Sherwood luôn cho đáp án đúng, nhưng dùng ngẫu nhiên để phá mối liên hệ giữa dữ liệu xấu và hành vi xấu của thuật toán xác định. Khi một thuật toán có độ phức tạp trường hợp xấu chênh lệch nhiều so với trung bình, ta có thể đưa ngẫu nhiên vào để làm cho trường hợp xấu không còn gắn chặt với một kiểu dữ liệu cụ thể. QuickSort ngẫu nhiên là ví dụ điển hình.

Thuật toán gia tăng ngẫu nhiên trong phần kế tiếp chính là một ứng dụng của tinh thần Sherwood. Ngoài ra, nhiều bài hình học cũng dùng được các tư tưởng điều chỉnh có ngẫu nhiên, như trong luận văn đội tuyển quốc gia 2006 của Tang Wenbin. Ví dụ bài Ural 1578 *Mammoth Hunt*: cho n điểm trên mặt phẳng, cần tìm một đường gấp khúc gồm $n - 1$ đoạn nối các điểm sao cho hai đoạn liên tiếp tạo góc nhọn. Một cách làm tự nhiên là sinh một đường gấp khúc bất kỳ, rồi điều chỉnh theo quy tắc cho tới khi thỏa điều kiện.

2 Thuật toán gia tăng ngẫu nhiên

2.1 Từ gia tăng tới gia tăng ngẫu nhiên

Tư tưởng của thuật toán gia tăng giống quy nạp toán học: biến bài toán kích thước n thành bài toán kích thước nhỏ hơn đúng một đơn vị. Sau khi giải bài con, ta thêm đối tượng hiện tại và cập nhật đáp án.

Ký hiệu S_i là tập gồm i đối tượng đầu tiên. Nếu đã có đáp án của S_i , ta thêm đối tượng thứ $i + 1$ để thu được đáp án của S_{i+1} . Cách viết này rất gọn và áp dụng được cho nhiều bài hình học.

Nhược điểm là thứ tự đầu vào có thể làm thuật toán phải sửa đáp án quá nhiều lần. Vì vậy ta thường xáo trộn ngẫu nhiên thứ tự xử lý trước khi chạy. Một thuật toán xáo trộn tuyến tính là Fisher–Yates:

```
for i = 2..n:
    swap(O[i], O[random(1..i)])
```

Sau bước này, $n!$ thứ tự đều có xác suất như nhau. Phần khó còn lại là phân tích kỳ vọng thời gian trên thứ tự ngẫu nhiên đó.

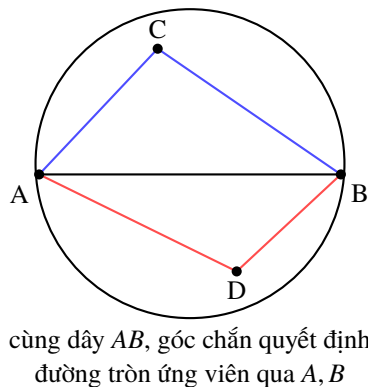
2.2 Ví dụ dẫn nhập: đường tròn bao nhỏ nhất

Cho n điểm trên mặt phẳng, cần tìm đường tròn nhỏ nhất chứa tất cả điểm. Bài viết dùng ví dụ này để phân tích tính chất của thuật toán gia tăng ngẫu nhiên, bắt đầu từ phương pháp góc chắn.

2.2.1 Phương pháp góc chắn

Trừ trường hợp chỉ có một điểm, đường tròn bao nhỏ nhất luôn có ít nhất hai điểm nằm trên biên. Ta có thể thử chọn hai điểm A, B . Với một điểm C nằm cùng phía với cung, góc $\angle ACB$ phản ánh kích thước đường tròn qua dây AB : với cùng dây AB , góc chắn càng nhỏ thì bán kính càng lớn.

Vì vậy với mỗi cặp A, B , ta xét các điểm ở hai phía của đường thẳng AB , tìm góc nhỏ nhất ở mỗi phía. Khi hai góc cực trị phù hợp, đường tròn ngoại tiếp ba điểm tương ứng có thể bao phủ toàn bộ tập điểm; trường hợp bốn điểm đồng viên xuất hiện khi hai góc cân bằng. Cách làm xác định này phải xét nhiều cặp và nhiều điểm, nên có độ phức tạp cao.



Hình 2: Sơ đồ phương pháp góc chắn: chọn dây AB , rồi xét các điểm không chế ở hai phía.

2.2.2 Cải tiến bằng gia tăng ngẫu nhiên

Gọi $P_i = \{p_1, p_2, \dots, p_i\}$, và $D(P_i)$ là đường tròn bao nhỏ nhất của P_i . Khi thêm điểm p_{i+1} , có hai trường hợp:

- nếu $p_{i+1} \in D(P_i)$, đáp án không đổi;
- nếu $p_{i+1} \notin D(P_i)$, thì p_{i+1} chắc chắn nằm trên biên của $D(P_{i+1})$.

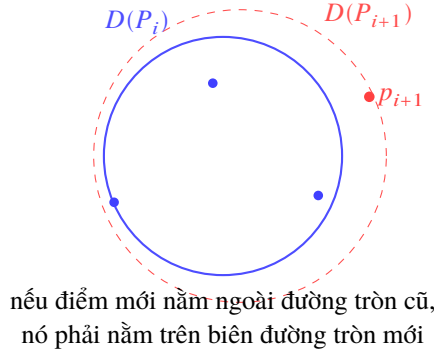
Mệnh đề thứ hai nhìn rất tự nhiên nhưng chứng minh không hoàn toàn hiển nhiên; phụ lục B ghi lại lập luận của bản gốc.

Từ định lý trên, khi thêm một điểm mới nằm ngoài đường tròn hiện tại, ta dựng lại đường tròn với điểm mới bị buộc nằm trên biên. Trong phiên bản dùng phương pháp góc chắn, ta chỉ cần xét các điểm đã xuất hiện trước đó cùng với điểm mới để tìm đường tròn bao mới.

Nếu không xáo trộn, độ phức tạp vẫn có thể xấu. Sau khi xáo trộn, ta xét biến ngẫu nhiên X_i , trong đó $X_i = 1$ nếu điểm thứ i làm thay đổi đường tròn hiện tại, và $X_i = 0$ nếu không. Chi phí ở bước i chỉ lớn khi $X_i = 1$. Nhờ tuyến tính của kỳ vọng, kỳ vọng tổng thời gian bằng tổng kỳ vọng của các chi phí từng bước, không cần các biến X_i độc lập.

Xét ngược lại: trong đường tròn bao của i điểm, số điểm biên quyết định đường tròn nhiều nhất là ba. Do thứ tự xử lý ngẫu nhiên, xác suất điểm thứ i rơi vào nhóm quyết định chỉ cỡ $\mathcal{O}(1/i)$. Vì thế số lần cập nhật đắt giá giảm mạnh theo kỳ vọng. Với cách dựng lại dựa trên góc chắn trong bài viết, độ phức tạp giảm từ $\mathcal{O}(n^3)$ xuống $\mathcal{O}(n^2)$ kỳ vọng; nếu tiếp tục dùng cùng định lý ở phần dựng lại, có thể đạt thuật toán tuyến tính kỳ vọng kinh điển.

Điểm quan trọng là chữ “kỳ vọng” ở đây xét theo thứ tự xáo trộn, hoàn toàn không giả định đầu vào được sinh ngẫu nhiên.



Hình 3: Gia tăng ngẫu nhiên cho đường tròn bao nhỏ nhất.

2.3 *Expensive Drink*: quy hoạch tuyến tính ba chiều

Expensive Drink có nội dung như sau. Em gái trộn nước, cà phê, rượu và đường thành các đồ uống. Giá đơn vị của bốn nguyên liệu là c_1, c_2, c_3, c_4 , trong đó nước rẻ nhất và rượu đắt nhất:

$$0 \leq c_1 \leq c_2 \leq c_3.$$

Nếu một đồ uống dùng lượng a_1, a_2, a_3, a_4 , giá của nó là

$$a_1c_1 + a_2c_2 + a_3c_3 + a_4c_4.$$

Ta biết n đồ uống cũ: mỗi đồ uống cho a_1, a_2, a_3 và tổng giá p ; đồng thời biết phần giá do đường tạo ra, tức a_4c_4 , luôn nằm trong đoạn $[L, R]$. Với một đồ uống đặc biệt chỉ cho a_1, a_2, a_3 , cần tìm giá lớn nhất có thể. Dữ liệu có $n \leq 100$, khoảng 2000 bộ test và giới hạn thời gian 2 giây.

Đặt x, y, z lần lượt thay cho c_1, c_2, c_3 . Phần đường không cần xuất hiện tường minh nữa, vì với mỗi đồ uống cũ ta có

$$L \leq p_i - a_{i1}x - a_{i2}y - a_{i3}z \leq R.$$

Cùng với các ràng buộc thứ tự $0 \leq x \leq y \leq z$, bài toán trở thành một quy hoạch tuyến tính trong không gian ba chiều:

$$\text{maximize } a_1x + a_2y + a_3z$$

dưới $2n + 6$ ràng buộc tuyến tính. Trong thi lập trình, gặp bài như vậy không nhất thiết phải viết simplex hay nội điểm.

2.3.1 Thuật toán thô

Trước hết xét vị trí của nghiệm tối ưu. Có vài khả năng: hệ vô nghiệm; nghiệm nằm ở giao của ba mặt phẳng; nghiệm nằm trên cạnh chung của hai mặt phẳng; nghiệm nằm trên cả một mặt; hoặc giá trị mục tiêu không bị chặn. Trường hợp không bị chặn có thể phát hiện bằng cách thêm một hộp rất lớn và xem nghiệm có rơi lên biên hộp hay không.

Do đó nếu bài toán có nghiệm hữu hạn, ta có thể tìm được một ứng viên từ giao của ba mặt phẳng ràng buộc. Thuật toán thô liệt kê mọi bộ ba mặt phẳng, tính giao điểm, rồi kiểm tra toàn bộ ràng buộc còn lại. Chi phí có thể tới $\mathcal{O}(m^4)$, với m là số ràng buộc.

2.3.2 Cắt đoạn thẳng

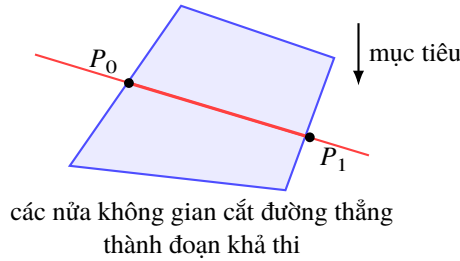
Vùng khả thi của quy hoạch tuyến tính là một tập lồi; trong bài này nó là một đa diện lồi. Thay vì liệt kê ba mặt phẳng, ta liệt kê hai mặt phẳng để lấy một đường thẳng. Mỗi ràng buộc còn lại cắt đường thẳng này thành một nửa đường thẳng; giao của tất cả chúng là một đoạn, một điểm, hoặc rỗng.

Trên một đoạn thẳng, hàm mục tiêu tuyến tính đạt cực trị ở một trong hai đầu mút. Nếu tham số hóa đoạn bằng

$$P(t) = P_0 + t(P_1 - P_0), \quad 0 \leq t \leq 1,$$

thì giá trị mục tiêu là một hàm bậc nhất theo t . Hệ số dương chọn P_1 , hệ số âm chọn P_0 , còn hệ số bằng không thì mọi điểm trên đoạn cho cùng giá trị.

Thuật toán này không bỏ sót giao của ba mặt phẳng: nếu giao điểm đó là đầu mút của đoạn thì đã được xét; nếu nằm bên trong đoạn thì theo tính tuyến tính nó không thể tốt hơn đầu mút tốt nhất; nếu không nằm trong đoạn thì nó không thỏa một ràng buộc nào đó. Khi cài đặt, ta dùng phương trình tham số của đường thẳng để biến mỗi phép cắt thành cập nhật khoảng giá trị của tham số. Chi phí tổng thể là $\mathcal{O}(m^3)$.



Hình 4: Hình phẳng hóa ý tưởng cắt đoạn: trên đoạn khả thi, chỉ hai đầu mút có thể tối ưu.

2.3.3 Gia tăng ngẫu nhiên

Thuật toán cắt đoạn vẫn chưa dùng mối liên hệ giữa nghiệm sau i ràng buộc và nghiệm sau $i + 1$ ràng buộc. Gọi C_i là giao của i nửa không gian đầu tiên và v_i là nghiệm tối ưu trên C_i . Khi thêm nửa không gian h_{i+1} , có hai trường hợp:

- nếu $v_i \in h_{i+1}$, nghiệm tối ưu không đổi;
- nếu $v_i \notin h_{i+1}$, thì nghiệm tối ưu mới, nếu tồn tại, phải nằm trên mặt phẳng biên của h_{i+1} .

Hai mệnh đề đều chứng minh được bằng phản chứng. Trường hợp thứ hai làm giảm bài toán xuống mặt phẳng biên mới, rồi giải bằng cắt đoạn.

Nếu xử lý theo thứ tự dữ liệu gốc, độ phức tạp vẫn có thể là $\mathcal{O}(m^3)$. Nhưng sau khi xáo trộn ngẫu nhiên các nửa không gian, một ràng buộc mới chỉ có xác suất cỡ $\mathcal{O}(1/i)$ để trở thành ràng buộc quyết định nghiệm tối ưu của i ràng buộc đầu. Tổng chi phí kỳ vọng giảm còn $\mathcal{O}(m^2)$ cho cách cài đặt ba chiều này.

Tác giả nhận xét rằng trên dữ liệu của bài, thuật toán này còn chạy nhanh hơn simplex. Về lý thuyết, quy hoạch tuyến tính số chiều cố định có thể tiếp tục giảm chiều để đạt tuyến tính theo số ràng buộc: mỗi lần nghiệm cũ bị loại, ta chiếu vector mục tiêu lên mặt phẳng mới và biến các nửa không gian còn lại thành nửa mặt phẳng để giải bài toán hai chiều. Tuy nhiên các phép chuyển hệ tọa độ, xoay mặt phẳng và hằng số ẩn làm cách đó không đáng dùng trong bối cảnh thi. Mục tiêu của ngẫu nhiên hóa ở đây là làm ít mà được nhiều, biến một cách viết tự nhiên thành một thuật toán đủ nhanh.

3 Mô phỏng tôi luyện

3.1 Nguyên lý và mô hình

Trong mắt nhiều thí sinh, bài hình học thường đồng nghĩa với mã nguồn dài. Ngay cả với những cấu trúc kinh điển, lượng mã lớn cũng khiến người viết dễ ngại. Mô phỏng tôi luyện là một meta-heuristic đơn giản và linh hoạt, có thể phát huy tác dụng trong một lớp bài tìm điểm xa nhất, gần nhất, gần thứ k , hoặc tối ưu tổng khoảng cách.

Ý tưởng xuất phát từ quá trình tôi luyện vật rắn: nung tới nhiệt độ cao, rồi làm nguội từ từ để hệ tiến tới trạng thái năng lượng thấp. Theo tiêu chuẩn Metropolis, khi nhiệt độ là T , một biến đổi làm năng lượng tăng ΔE vẫn có thể được chấp nhận với xác suất tỉ lệ

$$\exp\left(-\frac{\Delta E}{kT}\right),$$

trong đó k là hằng số Boltzmann. Trong thuật toán, ta thường hấp thụ hằng số vào tham số nhiệt độ và dùng dạng $\exp(-\Delta/T)$.

Mô hình tổng quát:

1. Chọn nhiệt độ ban đầu T đủ lớn, trạng thái ban đầu S , và số lần lặp L tại mỗi mức nhiệt.
2. Lặp L lần: sinh trạng thái mới S' .
3. Tính $\Delta = C(S') - C(S)$, với C là hàm đánh giá.
4. Nếu S' tốt hơn thì nhận; nếu xấu hơn thì nhận với xác suất $\exp(-\Delta/T)$.
5. Nếu thỏa điều kiện dừng thì xuất nghiệm hiện tại hoặc nghiệm tốt nhất.
6. Giảm T , rồi lặp lại.

3.2 Run Away

Run Away cho một hình chữ nhật kích thước $X \times Y$ và tối đa 1000 lỗ bẫy. Cần tìm điểm trong phòng sao cho khoảng cách tới lỗ gần nhất là lớn nhất; tọa độ xuất ra làm tròn đến một chữ số thập phân.

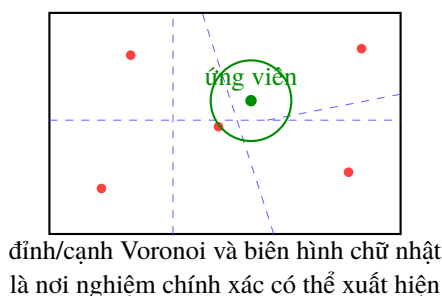
Lời giải hình học chính xác liên quan tới Voronoi diagram. Ứng viên tối ưu có thể là tâm đường tròn qua ba lỗ, giao của đường trung trực hai lỗ với biên hình chữ nhật, hoặc một trong bốn góc hình chữ nhật. Nói cách khác, nghiệm nằm ở đỉnh Voronoi, giao của cạnh Voronoi với biên, hoặc góc. Tuy nhiên xây dựng Voronoi bằng Fortune, Delaunay hoặc chia để trị nửa mặt phẳng đều khá dài.

Vì yêu cầu độ chính xác không quá khắt khe, ta có thể nhìn bài toán theo hướng mô phỏng tôi luyện:

- trạng thái S là một điểm trong hình chữ nhật;
- hàm đánh giá là khoảng cách từ S tới lỗ gần nhất;
- tham số nhiệt độ được hiểu là độ dài bước Δ ;
- trạng thái mới được sinh bằng cách chọn ngẫu nhiên một điểm trên vòng tròn tâm S , bán kính xấp xỉ Δ ;
- sau mỗi vòng, giảm Δ cho tới khi đủ nhỏ.

Để tránh phụ thuộc vào một điểm khởi đầu, ta dùng một tập ứng viên ban đầu đặt theo lưới trong hình chữ nhật, sau đó điều chỉnh song song tất cả ứng viên và lấy nghiệm tốt nhất cuối cùng. Bản gốc dùng $p = 10$ điểm theo mỗi chiều và $L = 30$ lần thử tại mỗi mức bước, đủ vượt qua toàn bộ dữ liệu kiểm thử.

Phân tích trực giác như sau: nghiệm tối ưu có khoảng trống lớn quanh nó, nên trong vùng Delaunay tương ứng, xác suất có ứng viên tiến gần tới nghiệm là không nhỏ. Khi bước điều chỉnh đã nhỏ, nếu điểm hiện tại nằm gần một cạnh Voronoi, các hướng làm tăng khoảng cách tới lỗ gần nhất chiếm một cung đáng kể. Với nhiều lần thử ở cùng một mức Δ , xác suất có ít nhất một lần đi đúng hướng tăng nhanh. Bản gốc còn chỉ ra rằng nếu cần độ chính xác cao hơn, có thể bổ sung các hướng song song với cạnh Voronoi; trong ba ví dụ của bài, điều chỉnh ngẫu nhiên thông thường đã đủ.



Hình 5: Sơ đồ ý tưởng *Run Away*: mô phỏng tôi luyện thay cho việc dựng Voronoi đầy đủ.

3.3 *Empire Strikes Back*

Empire Strikes Back cho một quốc gia hình tròn bán kính R , tâm $(0, 0)$. Có $N \leq 300$ nhà máy vũ khí hóa học tại các điểm (X_i, Y_i) . Mỗi quả bom rơi đúng vào một nhà máy và có cùng bán kính sát thương. Cần chọn bán kính nhỏ nhất sao cho hợp các hình tròn tâm tại nhà máy phủ kín toàn bộ quốc gia. Đáp án cần in ít nhất năm chữ số sau dấu thập phân.

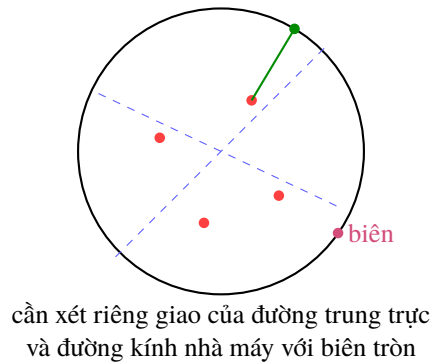
Bài này tương tự *Run Away*: ta cần tìm điểm trong đĩa quốc gia có khoảng cách tới nhà máy gần nhất lớn nhất. Khác biệt chính là biên hình tròn. Nếu chỉ dùng mô phỏng tôi luyện thô, khi ứng viên đã ở rất gần biên, một bước ngẫu nhiên thường rơi ra ngoài miền hợp lệ; xác suất cải thiện rất thấp. Hơn nữa trong bài này sai số theo phương biên có thể cộng trực tiếp vào đáp án, khác với trường hợp biên chữ nhật ở ví dụ trước.

May mắn là các trường hợp tối ưu trên biên không khó xử lý riêng. Nếu nghiệm nằm trên đường tròn biên, nó thuộc một trong hai loại:

- giao của đường kính đi qua một nhà máy với đường tròn quốc gia;
- giao của một cạnh Voronoi, tức đường trung trực của hai nhà máy, với đường tròn quốc gia.

Số ứng viên tương ứng là tuyến tính và bình phương. Kiểm tra chúng có chi phí $\mathcal{O}(n^3)$ với hằng số nhỏ, không chiếm quá nhiều thời gian so với phần tìm kiếm chính. Sau khi bổ sung đặc biệt hóa biên này, mô

phỏng tôi luyện vượt qua toàn bộ dữ liệu chính thức; với dữ liệu biên tự tạo có độ chính xác cao, các ứng viên biên cũng giúp thuật toán ổn định hơn.



Hình 6: Ứng viên biên trong *Empire Strikes Back*.

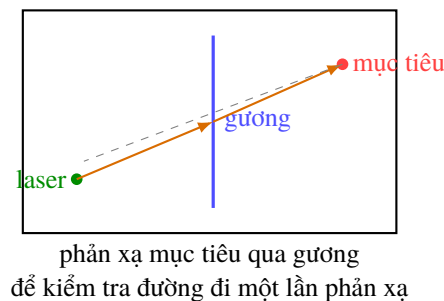
Tác giả cũng nhấn mạnh rủi ro khi dùng mô phỏng tôi luyện trong ACM: chỉ cần phân tích thiếu một loại dữ liệu, cả bài có thể sai. Tham số cũng quan trọng. Một lời giải ngẫu nhiên tốt trong thực tế vẫn cần hiểu rõ cấu trúc hình học để thêm các trường hợp đặc biệt cần thiết.

3.4 *Laser Tank*

Laser Tank là bài nộp đáp án ở CTSC. Cần chọn vị trí đặt bộ phát laser để bắn tới các mục tiêu trong môi trường có gương, với số lần phản xạ bị giới hạn. Điểm số được tính theo số bộ dữ liệu giải được.

Với $k = 0$, tia không phản xạ. Ta chỉ cần kiểm tra đoạn thẳng từ vị trí đặt laser tới mục tiêu có bị cản hay không. Với $k = 1$, có thể phản xạ một lần: ta phản xạ mục tiêu qua gương, biến đường gãy thành một đường thẳng, rồi kiểm tra giao điểm với gương và thứ tự cắt. Một tính chất hữu ích trong bản gốc là trường hợp có thể bắn được bằng một lần phản xạ thường bao hàm nhiều trường hợp không phản xạ, nên hàm đếm mục tiêu có thể dùng cho tìm kiếm ngẫu nhiên.

Từ thống kê điểm, chương trình cho $k = 0$ có tỉ lệ công sức/điểm rất tốt. Chương trình cho $k = 1$ dài hơn nhưng vẫn đáng viết nếu còn thời gian. Khi $k = 2$, bài toán phức tạp rõ rệt: phải tính hai lần phản xạ, kiểm tra thứ tự các giao điểm, đồng thời so sánh nhiều đường đi vì một lần phản xạ không nhất thiết tốt hơn hai lần phản xạ. Bản gốc đã viết thử và giải thêm được một bộ dữ liệu, nhưng cho rằng trong thi thật không đáng sửa lớn chương trình chỉ để lấy ít điểm bổ sung.



Hình 7: Sơ đồ hóa kiểm tra một lần phản xạ trong *Laser Tank*.

Kết luận của ví dụ này không phải là mô phỏng tôi luyện luôn cho nghiệm tối ưu, mà là trong bài nộp đáp án, một khung tìm kiếm ngắn, dễ thay đổi tham số và có thể chạy nhiều lần lấy kết quả tốt nhất thường rất có giá trị thực tế.

3.5 Tiểu kết

Mô phỏng tôi luyện có thể mở rộng cho nhiều bài cực đại, cực tiểu, gần thứ k , hoặc tối ưu tổng khoảng cách, đặc biệt khi giá trị hàm thay đổi liên tục theo vị trí. Không phải thí sinh nào cũng viết được farthest-point Voronoi hay k -order Voronoi trong thời gian thi, nhưng một lời giải tôi luyện được phân tích kỹ vẫn có thể cho kết quả đủ tốt. Nó kế thừa các ưu điểm chung của thuật toán ngẫu nhiên: thủ tục chính ngắn, cài đặt đơn giản, chạy nhanh, linh hoạt và dễ song song hóa.

Dù vậy, dữ liệu của các hệ thống như Ural khá chặt chẽ. Việc một thuật toán ngẫu nhiên vượt qua kiểm thử phản ánh chất lượng nghiệm khá tốt, nhưng không thay thế được phân tích. Với những chỗ bản thân mô phỏng để kẹt, như biên tròn của *Empire Strikes Back*, cần thêm lập luận hình học và xử lý riêng.

4 Kết luận

Trong những năm gần đây, hình học trong thi tin học ngày càng linh hoạt. Các bài không còn chỉ yêu cầu chép lại những cấu trúc cổ điển; nhiều bài đòi hỏi kết hợp mô hình, tối ưu và xử lý sai số. Thuật toán ngẫu nhiên, với ưu điểm đơn giản, nhanh, linh hoạt và dễ song song hóa, giúp cân bằng giữa thời gian, bộ nhớ và độ chính xác.

Tuy nhiên ngẫu nhiên hóa không thay thế nền tảng hình học. Muốn dùng nó tốt, ta vẫn phải hiểu rõ cấu trúc bài toán: điểm nào quyết định đường tròn bao nhỏ nhất, mặt phẳng nào quyết định nghiệm quy hoạch tuyến tính, biên nào làm mô phỏng tôi luyện để kẹt. Sự sáng tạo luôn xây trên nền tảng chắc. Hy vọng các tư tưởng trong bài viết mở thêm một hướng suy nghĩ khi gặp bài hình học.

Lời cảm ơn

Tác giả cảm ơn huấn luyện viên Liu Rujia đã hướng dẫn; cảm ơn bạn Yu Huacheng vì các cuộc thảo luận qua nhiều bài; cảm ơn nhóm tin học của Trường Trung học số 1 Trung Sơn vì nhiều năm huấn luyện và trao đổi; và cảm ơn các tài nguyên trên Internet.

References

- [1] Udi Manber, *Introduction to Algorithms: A Creative Approach*.
- [2] M. H. Alsuwailayel, *Algorithms: Design Techniques and Analysis*.
- [3] Liu Rujia và Huang Liang, *Nghệ thuật thuật toán và thi tin học*.
- [4] M. de Berg và cộng sự, *Computational Geometry: Algorithms and Applications*.
- [5] Tang Wenbin, *Bàn về tư tưởng điều chỉnh trong thi tin học*, luận văn đội tuyển quốc gia 2006.
- [6] Wang Dong, *Bàn về xây dựng và ứng dụng Voronoi diagram phẳng*, luận văn đội tuyển quốc gia 2007.
- [7] Gao Yihan, *Các phương pháp rời rạc hóa liên quan tới đường tròn*, luận văn đội tuyển quốc gia 2007.
- [8] Li Yuqian, *Quy hoạch tuyến tính trong thi tin học: cài đặt simplex gọn và hiệu quả*, luận văn đội tuyển quốc gia 2007.

A Ba bước lấy mẫu Monte Carlo

Bản gốc tóm tắt lấy mẫu Monte Carlo thành ba bước chính.

Mô tả hoặc xây dựng quá trình xác suất. Nếu bản thân bài toán đã có tính ngẫu nhiên, ta cần mô tả và mô phỏng đúng quá trình đó. Nếu bài toán là xác định, như tính tích phân xác định, ta phải tự xây dựng một quá trình xác suất sao cho một tham số của quá trình chính là nghiệm cần tìm.

Lấy mẫu từ phân bố đã biết. Sau khi có mô hình xác suất, ta cần sinh biến ngẫu nhiên theo các phân bố của mô hình. Trong máy tính, số ngẫu nhiên thường được tạo bằng công thức truy hồi, nên thực chất là giả ngẫu nhiên. Dù không phải ngẫu nhiên thật, các dãy này vượt qua nhiều kiểm định thống kê và đủ dùng như nguồn mẫu trong mô phỏng.

Xây dựng đại lượng ước lượng. Sau khi mô phỏng, ta chọn một biến ngẫu nhiên hoặc một thống kê mẫu làm ước lượng không chệch hoặc xấp xỉ tốt cho đại lượng cần tính. Nói cách khác, ta ghi nhận kết quả thí nghiệm mô phỏng và từ đó rút ra lời giải.

B Chứng minh định lý đường tròn bao

Gọi P là một tập điểm trên mặt phẳng, R là một tập điểm khác, có thể rỗng, và $R \subseteq P$. Xét các đường tròn phủ P và đi qua mọi điểm của R . Nếu tồn tại ít nhất một đường tròn như vậy, thì trong số chúng có một đường tròn nhỏ nhất duy nhất; ký hiệu là $D(P, R)$.

Tính duy nhất được chứng minh bằng phản chứng. Giả sử có hai đường tròn nhỏ nhất khác nhau D_0, D_1 cùng bán kính, tâm lần lượt x_0, x_1 . Lấy một điểm giao z của hai biên và xét họ đường tròn có tâm chạy liên tục trên đoạn x_0x_1 , vẫn đi qua z . Vì mọi điểm của P nằm trong giao $D_0 \cap D_1$, đường tròn ở giữa họ này vẫn phủ P và vẫn đi qua các điểm của R , nhưng bán kính nhỏ hơn. Mâu thuẫn với tính nhỏ nhất.

Từ đó suy ra hai mệnh đề dùng trong thuật toán:

- nếu $p \in D(P, R)$, thì $D(P \cup \{p\}, R) = D(P, R)$;
- nếu $p \notin D(P, R)$, thì p nằm trên biên của $D(P \cup \{p\}, R)$, hay tương đương

$$D(P \cup \{p\}, R) = D(P \cup \{p\}, R \cup \{p\}).$$

Mệnh đề thứ hai cũng dùng tính liên tục của họ đường tròn đi từ nghiệm cũ tới nghiệm mới: nếu nghiệm mới không đi qua p , trong quá trình biến đổi sẽ có một đường tròn nhỏ hơn vừa phủ P , vừa đi qua R , vừa đưa p lên biên, mâu thuẫn với định nghĩa nghiệm mới.

C Các phát biểu bài trong phụ lục gốc

Bản gốc kèm các phát biểu tiếng Anh của *Expensive Drink*, *Run Away* và *Empire Strikes Back*, cùng các gợi ý nhúng cho *Laser Tank*. Nội dung thiết yếu đã được dịch và lồng vào các phần ví dụ tương ứng ở trên. Các chương trình tham khảo dài không được chép lại theo quy tắc không dịch code templates; bản gốc cũng ghi rằng *Empire Strikes Back* gần với *Run Away* nên người đọc có thể tự viết, còn *Laser Tank* là bài nộp đáp án nên tham số và chương trình cần chỉnh theo từng bộ dữ liệu.

Ghi chú biên tập

Tệp nguồn là tài liệu Word cũ, trong đó nhiều công thức và hình được nhúng dưới dạng đối tượng OLE. Bản dịch này khôi phục các công thức rõ nghĩa từ văn cảnh, vẽ lại các sơ đồ chính bằng TikZ theo nội dung nguồn, và chuyển các đoạn phụ lục/problem statement quan trọng thành tiếng Việt. Những công thức bị mất hoàn toàn trong luồng văn bản được diễn giải bằng lời thay vì đoán ký hiệu chi tiết.